

Mini Project — Asset Tracking System #1

Project Overview

In this mini project, students will build a simple **Asset Tracking System** using **C# Console Applications** and **Object-Oriented Programming (OOP)** concepts.

The application will help a company keep track of its assets such as:

- Laptop Computers
- Desktop Computers
- Mobile Phones
- Tablets (optional extension)

The system should allow users to:

- Create assets
- Store them in collections (List<T>)
- Sort and display assets
- Detect end-of-life status
- Manage assets from different offices worldwide

Learning Goals

Students will practice:

- Classes & Objects
- Constructors
- Properties
- Inheritance
- Abstract Classes / Interfaces
- Lists and Collections
- Sorting
- DateTime operations

- Console formatting
 - Enums
 - Basic file handling (advanced level)
-

Business Rules

- All company assets have a lifespan of **3 years**
 - Assets nearing end-of-life should be highlighted:
 - **YELLOW** → less than 3 months remaining
 - **RED** → less than 6 months remaining
 - Assets belong to offices in different countries
 - Each office uses its own currency
-

Level 1 — Basic Classes & Objects

Task

Create a console application with classes representing company assets.

Example asset categories:

Computers

- MacBook Pro
- Lenovo ThinkPad
- Asus ZenBook

Mobile Phones

- iPhone
- Samsung Galaxy
- Nokia

Requirements

Each asset should contain:

Property	Example
Brand	Apple
Model	MacBook Pro
Purchase Date	2024-03-15
Price (USD)	1500
Price (Local)	15000 SEK
Office	Sweden
Asset Type	Laptop

Technical Requirements

- Use classes and objects
- Use constructors
- Create at least:
 - Computer

- MobilePhone
- Store objects inside a List<Asset>

Example Console Output

ASSET LIST

Type	Brand	Model	Purchase Date
Laptop	Apple	MacBook Pro	2024-03-15
Laptop	Lenovo	ThinkPad X1	2023-10-01
Phone	Samsung	Galaxy S23	2025-01-10
Phone	Apple	iPhone 15	2024-12-05



Level 2 — Sorting & End-of-Life Warning

Task

Extend the application to support:

- Sorting assets
 - Detecting assets near end-of-life
-

Requirements

Sort assets by:

1. Asset Type
2. Purchase Date

The system should:

- Display assets in sorted order
 - Highlight assets close to 3 years old
-

End-of-Life Rules

Status	Condition
YELLOW	Less than 3 months remaining
RED	Less than 6 months remaining

Suggestions

Use:

- DateTime
 - TimeSpan
 - LINQ sorting
-

Example Console Output

ASSET LIST

Type	Brand	Model	Purchase Date	Status
Laptop	Apple	MacBook Pro	2022-08-10	RED
Laptop	Asus	ZenBook	2023-01-05	YELLOW
Laptop	Lenovo	ThinkPad X1	2024-04-01	
Phone	Nokia	XR20	2022-09-01	RED
Phone	Samsung	Galaxy S23	2025-01-10	



Level 3 — Multi-Office & Currency Support

Task

Add office locations and currency support.

The company now has offices in:

- Sweden (SEK)
 - USA (USD)
 - Turkey (TRY)
-

Requirements

Each asset should belong to an office.

Example:

Office	Currency
Sweden	SEK
USA	USD
Turkey	TRY

Currency Conversion

Prices may initially be entered in USD and converted into local currency.

You may:

- Hardcode exchange rates
OR
 - Use an API (optional advanced challenge)
-

Sorting Requirements

Sort by:

1. Office
 2. Purchase Date
-

Example Console Output

ASSET LIST

Office	Type	Brand	Model	Price	Purchase Date	Status
Sweden	Laptop	Apple	MacBook Pro	16500 SEK	2022-08-10	RED
Sweden	Phone	Samsung	Galaxy S23	9500 SEK	2024-11-01	
Turkey	Laptop	Lenovo	ThinkPad X1	72000 TRY	2023-01-05	YELLOW
USA	Phone	Apple	iPhone 15	1200 USD	2025-02-01	



Level 4 — Advanced OOP Design

Task

Improve the application architecture using better OOP principles.

Requirements

Implement:

- Abstract base class OR interface
- Inheritance
- Polymorphism

Example structure:

```
Asset (abstract)
├─ Computer
├─ MobilePhone
└─ Tablet
```

Additional Features

Add:

- Unique Asset ID
- Asset Age Calculation
- Menu System

Example menu:

1. Add Asset
2. View Assets
3. Sort Assets
4. Search Asset
5. Exit



Example Console Output

```
=====
COMPANY ASSET TRACKING SYSTEM
=====

1. Add Asset
2. View Assets
3. Search Assets
4. Remove Asset
5. Exit

Select option:
```



Level 5 — File Management System (Optional - Hard Level)

Task

Instead of using only lists in memory, save and load assets from files.

This level introduces persistent storage without using a database.

Requirements

Students should:

- Save assets to a file
 - Load assets when the application starts
 - Update the file when assets are added or removed
-

Suggested Technologies

Students may use:

- JSON files
- CSV files
- Text files

Recommended:

- `System.Text.Json`
-

Features

- Auto-save asset list
 - Read assets from file on startup
 - Export asset report
 - Prevent duplicate asset IDs
-

Example File Structure

assets.json

JSON

```
[
  {
    "id": 1,
    "type": "Laptop",
    "brand": "Apple",
    "model": "MacBook Pro",
    "office": "Sweden",
    "price": 16500,
    "currency": "SEK"
  }
]
```

Example Console Output

```
Loading assets from file...
5 assets loaded successfully.

New asset added successfully.
Saving changes to assets.json...

Operation completed.
```

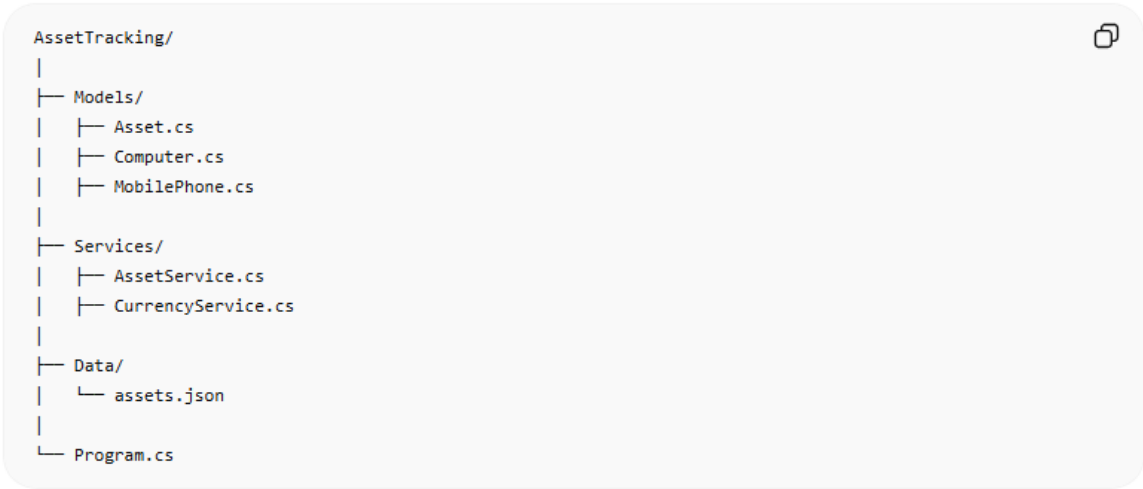
Optional Bonus Challenges

Students can additionally implement:

- Search by brand/model
- Remove asset
- Edit asset
- Pagination
- Colorful console UI
- Export to CSV
- Exception handling

- API-based currency conversion
- Unit tests

Suggested Folder Structure



```
AssetTracking/  
|  
├─ Models/  
|   ├─ Asset.cs  
|   ├─ Computer.cs  
|   └─ MobilePhone.cs  
|  
├─ Services/  
|   ├─ AssetService.cs  
|   └─ CurrencyService.cs  
|  
├─ Data/  
|   └─ assets.json  
|  
└─ Program.cs
```

Instructor Notes

This project is designed to gradually introduce students to:

- clean code practices
- scalable architecture
- real-world software structure
- progressive OOP concepts

Students should first focus on functionality before attempting advanced architecture improvements.